

Benchmark Survey

TL;DR:

This benchmark is specifically designed for an agent whose main task is to find and recommend freelancers.

The full benchmark contains four tracks. **Track A (Intent & Interaction)** evaluates whether the agent can understand the client's real needs, identify missing information, and correctly update its understanding during multi-turn conversations. **Track B (Planning & Tool Use)** evaluates whether the agent can independently decide what to do next, such as asking a question, checking client history, searching, viewing candidates, or checking candidate readiness, and whether it can use the right tools correctly. **Track C (Matching & Readiness)** evaluates whether the agent can select suitable freelancers by considering task fit, client preferences, candidate readiness, and reputation together. **Track D (Closed-loop Hiring Outcome)** further checks whether the recommended candidate can actually complete the task and meet the acceptance criteria.

Among the existing datasets, **Upwork_GPT** mainly supports the evaluation of real client needs and tool-use behavior. **UFS/UMA** supports multi-turn requirement clarification and state changes. **HAPIv2** provides real tasks, criterion-level acceptance criteria, and final work outcomes. The most important missing part is a **Bridge Dataset** that connects:

Client Request → Client History → Agent Trajectory → Candidate Pool → Readiness → Candidate Selection → Actual Work Outcome

The final benchmark can be used in two ways. First, the four tracks can be tested separately to identify specific weaknesses of an agent. Second, complete episodes can be built to evaluate the full end-to-end process:

Understanding the Need → Making Decisions → Using Tools → Selecting Freelancers → Real Task Outcome

The main evaluation results should directly report scores for the four tracks. This provides a more detailed and more realistic evaluation of agents than simple

metrics such as Pass Rate, Hire Rate, or NDCG.

1. Agent

Based on Upwork's *Client Matching Big Bet Q2/Q3 '26* plan, the core goal of Stage 2 is to design an agent like this:

Move from a passive matching system ("user types a Query → the system returns search results") to an agent that understands who the client is, understands what the client wants to do right now, predicts what they will need next, and actively uses matching tools to complete the hiring task.

Upwork defines Stage 2 as **"Intelligence for What's Next."** The document clearly states that passive Ranking cannot keep up with an Agentic World. In the future, Upwork needs to build three types of basic capabilities: **Intent, Readiness, and Representation**. In other words, the system needs to understand not only "what the user typed just now," but also "who this client is," "whether a candidate is a good fit to contact right now," and "what the client may need next."

So this document defines the Stage 2 Agent as:

Intent-Aware Recruiting Agent

Intent-Aware Recruiting Agent

This task is not simply about "finding a freelancer." It is about:

Continuously understanding the client's hiring goal and project state, keeping a live picture of both the client and the talent, deciding the most reasonable hiring action right now, calling Upwork's search and matching tools, and helping the client make the next hiring decision through multi-turn conversation.

The full core flow should be:

User goal / current conversation
↓

```
Understand hiring intent
  ↓
Read the client's history and current state
  ↓
Judge whether the information is enough
  ↓
Decide the next action
  ↓
Call tools such as Search / Match / Readiness
  ↓
Compare, filter, and explain candidates
  ↓
Give the user a suggestion or keep asking
  ↓
Update the state with new information
  ↓
Move to the next hiring decision
```

From a system-capability view, the Stage 2 Agent needs at least six core capabilities:

1. **Conversational Intent Understanding**
2. **Persistent Client Representation**
3. **Freelancer Readiness Awareness**
4. **Sequence-aware Next-step Prediction**
5. **Tool Planning and API Orchestration**
6. **Multi-turn State Tracking**

2. Benchmark

Given the Agent design, our Benchmark should fully evaluate this whole flow:

```
Understand the client
  ↓
Find the information gap
  ↓
Decide whether to ask or to search
  ↓
Use the history and state
  ↓
Call the right tool
```

```
↓  
Consider the talent's real-time state  
↓  
Recommend the right person  
↓  
Whether the final hire produces high-quality work
```

Upwork's Agent needs to build three types of capability: intent, readiness, and representation. Stage 2 also clearly requires sequence-aware recommendation, conversational query understanding, MCP retrieval, and Match/Search APIs that the Agent can call.

So the target of the benchmark must move from:

```
Query → Ranking
```

up to:

```
Client Goal  
→ Understanding  
→ State  
→ Planning  
→ Tool Use  
→ Matching  
→ Hiring Outcome
```

2.1 Datasets We Already Have

HAPIv2: Very rich results, but no hiring process

- `jobs` has 145 real Web / Mobile / Software Development tasks;
- Each task has specific `acceptance_criteria`;
- On average about 6 acceptance criteria per task;
- The main evaluation chain has 104 tasks;
- `agent_traces` and `evals_turn0` cover 104 tasks × 3 Agents;
- There are also `evals_turn1` and `evals_turn2`;

- After a failure, the work can be edited and submitted again;
- `submission_feedback` can be tied to a specific Criterion;
- The most natural evaluation unit is:

Task × Agent × Turn × Criterion

It naturally has:

```
Real tasks
+
Clear acceptance criteria
+
Real deliverables
+
Item-by-item scoring
+
Failure diagnosis
+
Edit and retry
+
Final success / failure
```

But it does not have:

```
Who the Client is
Who the client hired before
Why they want to hire
What the candidate pool is
Why the Agent chose this person
Which search tool the Agent called
Whether it asked clarifying questions
What the hiring process was
```

So HAPIv2 cannot evaluate a Recruiting Agent on its own.

Upwork_GPT: Rich process, but not enough final results

It contains:

```
conversation / session
tool_name
structured inputs
query
additional_context
location
hourly_rate
search
view
hire
```

It also contains user-behavior flows like:

```
Search
Search → View
Search → View → Hire
```

It is a good fit for answering:

- | Did the Agent understand the Query correctly?
- | Did the Agent call the right Tool?
- | Were the Tool arguments reasonable?
- | Was the order of actions reasonable?

But the main problem right now is:

- | Many records still stop at Search.

The full chain:

```
Goal
→ Search
→ View
→ Invite
→ Hire
→ Work Outcome
```

is rare.

So it is hard for it to tell us:

| Is the person the Agent recommended actually good in the end?

UFS / UMA data: A great fit for evaluating Clarification

It already defines a kind of:

Step-by-step Refinement

For example:

Turn 1:
Location

Turn 2:
Experience / Badge

Turn 3:
JSS

And it stresses inside:

Handle only one refinement dimension at a time

This is exactly what we can use to evaluate the Stage 2 Agent's:

Clarification Policy
State Tracking
Constraint Accumulation
Context Consistency

But its problem is:

| Right now the training data is mostly synthetic.

So we cannot treat it directly as "real user-behavior Ground Truth."

2.2 The Core Research Question of the Benchmark

We should not define the research question as:

| How accurately can an agent recommend freelancers?

Because this question is too much like a traditional recommender system. A better core question is:

| **Can an agent understand a client's evolving hiring goal, choose the right next action, use matching tools effectively, and produce a high-quality hire?**

This question has four layers:

```
Layer 1
Did it understand correctly?

Layer 2
Did it act correctly?

Layer 3
Did it recommend correctly?

Layer 4
Was the final hiring result good?
```

These four layers are exactly what most current benchmarks do not evaluate at the same time. This is also what makes it different from a traditional recommender system.

2.3 The Basic Unit of the Benchmark

Episode × Agent × Turn × Decision Criterion

One Episode is one complete hiring task.

For example:

Episode #183

Client:
AI startup

Existing team:
Designer
Frontend Developer

Current project:
AI SaaS prototype

Hidden goal:
Production deployment

Initial user message:
"We are building an AI product
and need someone to help."

Candidate pool:
20 freelancers

Available tools:
Search
View
Client History
Readiness
Reputation
Match

Maximum turns:
8

Then we record:

Turn 1
Agent interprets intent

Turn 2
Agent asks clarification

Turn 3
User provides new information

Turn 4
Agent retrieves history

Turn 5

Agent searches

Turn 6

Agent checks readiness

Turn 7

Agent compares candidates

Turn 8

Agent recommends

Finally we connect:

Chosen Candidate

↓

Actual Work

↓

HAPI Acceptance Criteria

↓

Downstream Quality

2.4 Benchmark Architecture Design

We design the Benchmark as four Tracks that connect to each other but can also run on their own.

Track A: Intent & Interaction

Evaluate whether the Agent truly understands the client

Mainly uses:

Upwork_GPT

+

UFS / UMA

+

New labels

Evaluation metrics:

```
Intent Understanding
Constraint Extraction
Missing Information Detection
State Tracking
Clarification Quality
```

Tasks include:

A1. Explicit Intent

The user clearly says:

| I need a senior React developer in Australia.

The Agent should extract correctly:

```
role = React Developer
seniority = Senior
location = Australia
```

A2. Ambiguous Intent

The user:

| I need help launching my AI product.

The Agent should not Search right away.

The correct behavior should be:

```
ASK_CLARIFICATION
```

A3. Long-form Intent

For example:

We have already built a prototype using PyTorch. We now want to serve it to customers through our web application and expect a few thousand requests per day.

The Agent should infer:

```
Current state:
Prototype exists

Target:
Production deployment

Likely role:
MLOps / ML Infrastructure

Missing:
Cloud provider
Deployment stack
```

A4. Changing Intent

```
Turn 1:
Need Python engineer

Turn 3:
Actually we already have backend support

Turn 5:
The main issue is deployment
```

Evaluate whether the Agent:

```
Updates the state
```

instead of continuing to search for the original Python Engineer.

Track B: Planning & Tool Use

In the deck, the Stage 2 Roadmap clearly asks to turn Search/Match into Agent-consumable primitives, instead of making every product re-implement the search logic.

The Agent's Actions can be defined as:

```
ASK
RETRIEVE_HISTORY
SEARCH
VIEW
CHECK_READINESS
CHECK_REPUTATION
MATCH
COMPARE
REFINE
RECOMMEND
STOP
```

A typical test

The user:

| Find someone similar to the developer I hired last year.

Wrong:

```
SEARCH("developer")
```

Correct:

```
RETRIEVE_HISTORY
  ↓
IDENTIFY_PREVIOUS_HIRE
  ↓
SEARCH_SIMILAR
  ↓
CHECK_READINESS
```

↓
RECOMMEND

Evaluate:

Tool Selection
Tool Ordering
Argument Accuracy
Redundant Calls
Error Recovery

Track C: Matching & Readiness

This is the part that connects the recommender system and the Agent.

The Roadmap clearly asks the Freelancer Readiness Model to estimate near-real-time readiness from behavior such as bids, messages, and alert interactions. At the same time, the Client–Talent Intent Embedding should use past behavior and talent alignment to form a client representation that carries across sessions.

So a Candidate's evaluation cannot only be:

Skill Match

It should be:

Candidate Utility

=
Task Fit
×
Client Fit
×
Readiness
×
Trust

For example:

Candidate	Task Fit	Readiness	Final result
A	0.97	0.15	Should not be the first recommendation
B	0.91	0.88	Most reasonable
C	0.82	0.95	Can be a backup

Traditional Ranking:

A > B > C

Agentic Matching:

B > C > A

Metrics

We can keep the traditional recommendation metrics:

NDCG@K
Recall@K
Hit Rate

But we must add:

Readiness-aware Utility
Constraint Satisfaction
Personalization

Track D: Closed-loop Hiring OutcomeCore question

Did the person the Agent recommended actually do the task well in the end?

This is the biggest research innovation of the Benchmark.

Traditional Recommendation Benchmarks stop at the recommendation.

Traditional hiring data usually stops at the Hire step.

But we can go all the way to:

```
Recommend
  ↓
Hire
  ↓
Work
  ↓
Acceptance Criteria
  ↓
Real Task Outcome
```

Our Agent will really evaluate:

The real downstream value that the recommendation produces.

For example:

```
Criterion 1: Pass
Criterion 2: Pass
Criterion 3: Fail
Criterion 4: Partial pass
Criterion 5: Pass
```

Compute:

```
Final Work Quality = 0.76
```

Metrics

```
Acceptance Criteria Satisfaction
Overall Task Quality
Final Success
Decision Regret
```


2.5 How the Four Tracks Connect

The full flow should be:



So the full Benchmark asks:

Can the Agent understand the client → make the right decisions → pick the right talent → produce a good, real task result.

3. Where Each of the Three Existing Datasets Fits

Data	Track A	Track B	Track C	Track D
Upwork_GPT	✓	✓	Partial	
UFS / UMA	✓	Partial		
HAPIv2			Partial	✓
New Bridge Dataset		✓	✓	✓

```
Upwork_GPT
  ↓
Real client needs and Tool scenarios
  ↓
Track A + Track B

UFS / UMA
  ↓
Multi-turn changes in needs
  ↓
Track A

Candidate Dataset
  ↓
Candidate Profile + Readiness
  ↓
Track C

HAPIv2
  ↓
Real task execution results
  ↓
Track D
```

4. The Data We Are Missing

We are missing a

Bridge Dataset

Its job is to connect the four Tracks.

Each complete Episode needs:

```
Client Request
+
Client History
+
Conversation State
+
Available Tools
+
Candidate Pool
+
Candidate Readiness
+
Candidate Actual Outcome
```

Full structure:

```
Client
↓
Track A
Understand the need
↓
Track B
Choose the action
↓
Track C
Choose the Candidate
↓
Track D
Check the real Outcome
```

5. Version Linev1: Four separate Track Benchmarks

Track	Suggested size
A Intent & Interaction	300–500

Track	Suggested size
B Planning & Tool Use	300–500
C Matching & Readiness	200–300
D Closed-loop Outcome	100–300

This lets us diagnose:

Where exactly is the Agent weak?

For example:

Agent A

Track A: 90
Track B: 55
Track C: 80
Track D: 76

This means:

Understanding is very good, and matching is fine too, but Planning / Tool Use is very weak.

V2: Integrated Benchmark

Then we design about:

200–300 complete Episodes

Run in full:

Track A
↓
Track B
↓
Track C

↓
Track D

That is:

Real need
→ Multi-turn understanding
→ Autonomous Tool Use
→ Candidate Selection
→ Real Work Outcome

6. Final Leaderboard

Agent	Track A Understanding	Track B Decision	Track C Matching	Track D Outcome
Agent A	86	72	80	75
Agent B	78	91	85	84
Agent C	90	65	76	70